Year 3 CSE Assignment

Name: Nelson RUTWAZA

Registration Number: 223026976

Course: Mobile Development

Date: 10-Mar-26

State Management in Flutter

Name : Nelson RUTWAZA
Reg: 223026976
Course: Mobile Development
Date : 10 - Mar - 26

UR · YEAR 3 · CSE

STATE MANAGEMENT IN FLUTTER ASSIGMENT.

0 Explanation of flutter state management Techniques.

- **Provider**

This is the most widely used state management solutions in flutter. It is built on top of Flutter's inherited Widget mechanism and allows data to be shared efficiently access different parts of widget tree.

provider approach improves performance and keeps the code organised because the business logic is separated from the user interface.

- **Riverpod**

This is an improved version of provider, but it does not rely on the flutter widget tree to access state. instead it uses more flexible system where providers can be accessed from anywhere in the application.

- **Bloc (Business Logic Component)**

Bloc is a state management pattern that focuses on separating the business logic from the presentation layer. It is based on the concept of events and states.
In bloc pattern the UI sends events to the Bloc, the Bloc processes those events and produces new states, the UI then rebuilds based on updated state.

- **GetX**

This is a lightweight and powerful Flutter framework that provides state management, dependency injection, and route management in single package.
Developers can manage application state using reactive variables, which automatically update the UI whenever the data changes.

CamScanner

② Table showing suitable Use Cases for each State Management

Situation	Provider	Riverpod	Bloc	GetX
Small Applications	Very suitable because it is simple & lightweight	Suitable but sometimes unnecessary	Not ideal due to complexity	Very suitable because it's fast and simple
Medium Applications	Good choice for moderate complexity	Very good due to flexibility and safety	Suitable if architecture is needed	Good choice for quick development
Large / Enterprise Applications	Can become difficult to manage	Very suitable for scalable architecture	Highly recommended	Possible but less structured
Team Projects	Acceptable but may lack strict structure	Good because of better dependency handling	Excellent because architecture is clear	Can be harder to maintain in large teams
Fast development	Moderate speed	Moderate	Slower due to structured setup	Very fast
Strict Architecture requirements	Limited	Good	Excellent	Limited

CamScanner

3. Key Steps for Using Provider in Flutter

>> Adding Dependency

The first step is adding the Provider package to the Flutter project. This is done by including it in the **pubspec.yaml** file.

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.0.0
```

After adding the dependency, the package is installed using:

```
flutter pub get
```

This allows the application to use Provider features.

>> Creating a State Class

The next step is creating a state class that holds the data and business logic. This class usually extends **ChangeNotifier**, which allows widgets to listen for updates.

Example:

```
import 'package:flutter/material.dart';

class CounterProvider extends ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}
```

The notifyListeners() method is very important because it informs the UI that the state has changed.

>> Providing the State

The state must then be provided to the widget tree using **ChangeNotifierProvider**.

Example:

```
ChangeNotifierProvider(
  create: (context) => CounterProvider(),
  child: MyApp(),
)
```

This makes the state available to all widgets inside the application.

>> Accessing the State

Widgets can access the provided state using the **Provider.of** method or the **Consumer widget**.

Example:

```
Provider.of<CounterProvider>(context)
```

or

```
Consumer<CounterProvider>(
  builder: (context, counter, child) {
    return Text(counter.count.toString());
  },
)
```

These methods allow widgets to read the current state.

>> Updating the State

When a function inside the state class changes the data, notifyListeners() is called.

Example:

```
context.read<CounterProvider>().increment();
```

This updates the value stored in the provider.

---

6. UI Rebuilding

Flutter follows a reactive UI approach. When the state changes and notifyListeners() is triggered, all widgets that are listening to that provider are automatically rebuilt.

For example, if a Text widget is displaying the counter value through a Consumer, it will rebuild and display the updated number whenever the counter changes.

This ensures that the user interface always reflects the latest application state.